PROGRAMA · TRAIN THE TRAINERS

Formación *AI Engineering.*

Prompting, contexto, seguridad, agent skills, templates del SDLC, tool calling con MCP, optimización y casos E2E (Hackathon)

DURACIÓN	FORMATO	COBERTURA
8 módulos × 5 h · 40 h	**Trainers BCP**	**5 grupos × 5 personas**

© 2026 NTT DATA, Inc.

Recorrido completo — Vista del programa

Ruta de aprendizaje, *5 días.*

Cinco jornadas de 09:00 a 18:00. Examen de entrada el Día 1; Gate G1 y Gate G2 al cierre de los Días 2 y 4.

1 Día

Prompting & Secure AI Engineering

- Prompting + Context Engineering
- Secure AI Engineering
- Optimización + Refactor
- Docs + Unit Testing

EXAMEN DE ENTRADA
Día 1 · 09:30–10:00

2 Día

Agent Skills, Templates & MCP

- Anatomía del Agent Skill
- Templates del nuevo SDLC
- Tool Calling
- MCP aplicado al SDLC

GATE G1 · EVALUACIÓN
Día 2 · 17:30–18:00

3 Día

SDD + Agentes

- OpenSpec, Spec Kit y Constitution
- Specify → Plan → Tasks → Validate
- Fundamentos de agentes
- Routing, Evaluator y HITL

4 Día

Agentic Engineering in Practice

- CrewAI: agentes por roles
- LangGraph: state y routing
- Harness Engineering
- Guardrails

GATE G2 · EVALUACIÓN
Día 4 · 17:30–18:00

5 Día

Hackathon AI Engineering

- Alinear y priorizar
- Construir en dos sprints
- Validar y congelar la versión
- Demo Arena y certificación

✓ CERTIFICACIÓN
Día 5 · 17:30–18:00

© 2026 NTT DATA, Inc.

Programa — Horario por jornada

Cinco jornadas, *un mismo ritmo.*

	DÍA 1 Prompting & Secure AI	DÍA 2 Skills, Templates & MCP	DÍA 3 SDD + Agentes	DÍA 4 Agentic Engineering	DÍA 5 Hackathon E2E
08:45	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break y registro
09:00	09:00–09:15 Introducción BCP	09:00–09:15 Apertura y repaso Día 1	09:00–09:15 Apertura y repaso Día 2	09:00–09:15 Apertura y repaso Día 3	09:00–09:15 Kick-off: reglas y reto
	09:15–09:45 **Examen de entrada**	09:15–10:00 S1 · Anatomía del Agent Skill	09:15–10:00 S1 · SDD: OpenSpec, Spec Kit	09:15–10:00 S1 · CrewAI: agentes por roles	09:15–09:45 1 · Alinear
10:00	09:45–10:30 M1 · Prompting + Context Eng.	10:00–10:50 Lab 1 · Skill básico versionado	10:00–10:50 Lab 1 · Definición técnica + contrato	10:00–11:00 Lab 1 · Crew API / BE / FE / reviewer	09:45–10:15 2 · Priorizar
	10:30–11:15 Lab 1 · Reescribir pedidos vagos	10:50–11:30 S2 · Templates del SDLC	10:50–11:30 S2 · Specify → Plan → Tasks	11:00–11:35 S2 · LangGraph: state y routing	10:15–10:30 Checkpoint 1 · Scope
11:00	11:15–11:50 M2 · Secure AI Engineering	11:30–12:30 Lab 2 · Skill desde template	11:30–12:30 Lab 2 · Spec-to-tasks	11:35–12:30 Lab 2 · Graph con estado	10:30–12:30 3 · Construir · Sprint 1
12:00	11:50–12:30 Lab 2 · Detección y corrección segura				
13:00	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo
14:00	13:30–14:15 M3 · Refactor y optimización + demo	13:30–14:20 S3 · Tool calling + demo	13:30–14:15 S3 · Fundamentos de agentes	13:30–14:00 S3 · Harness Engineering	13:30–14:45 3 · Construir · Sprint 2
	14:15–15:20 Lab 3 · Refactor guiado con Copilot	14:20–15:20 S4 · MCP: tools, resources y permisos	14:15–15:25 Lab 3 · Single-agent	14:00–15:15 Lab 3 · Harness del workflow	14:45–15:30 4 · Validar
15:00	15:20–16:00 M4 · Docs + unit testing asistido	15:20–16:00 Lab 3 · Skill + MCP en local	15:25–16:00 S4 · Routing, Evaluator y HITL	15:15–16:00 S4 · Guardrails	15:30–16:00 Checkpoint 2 · Freeze
16:00	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break
	16:15–17:45 Lab 4 · Docs y unit tests	16:15–17:15 Lab 3 (cont.) · Skill + MCP	16:15–17:45 Lab 4 · Agente + review gate	16:15–17:10 Lab 4 · Agente seguro	16:15–17:15 5 · Demo Arena · 2 salas × 5 equipos
17:00		17:15–17:30 Cierre del Día 2		17:10–17:30 Síntesis · puente al Día 5	17:15–17:30 Deliberación del jurado
18:00	17:45–18:00 Cierre · puente al Día 2	17:30–18:00 **Gate G1 · Evaluación**	17:45–18:00 Cierre · puente al Día 4	17:30–18:00 **Gate G2 · Evaluación**	17:30–18:00 **Cierre + Certificado**

© 2026 NTT DATA, Inc.

Contenido · Laboratorio · Apertura · cierre · Examen y Gates G1 / G2 · Certificación · Coffee · almuerzo

MCP.

Antes de iniciar... *Recordemos.*



DÍA 3

SDD & Fundamentos de *Agentes.*

Trazar el cambio antes de codificarlo, y poner al agente bajo control cuando lo escribe. Una jornada de 4 sesiones para pasar de *vibe coding* a un flujo auditable y a un agente gobernado con routing, evaluator y HITL.



AUDIENCIA

Trainers internos BCP

FORMATO

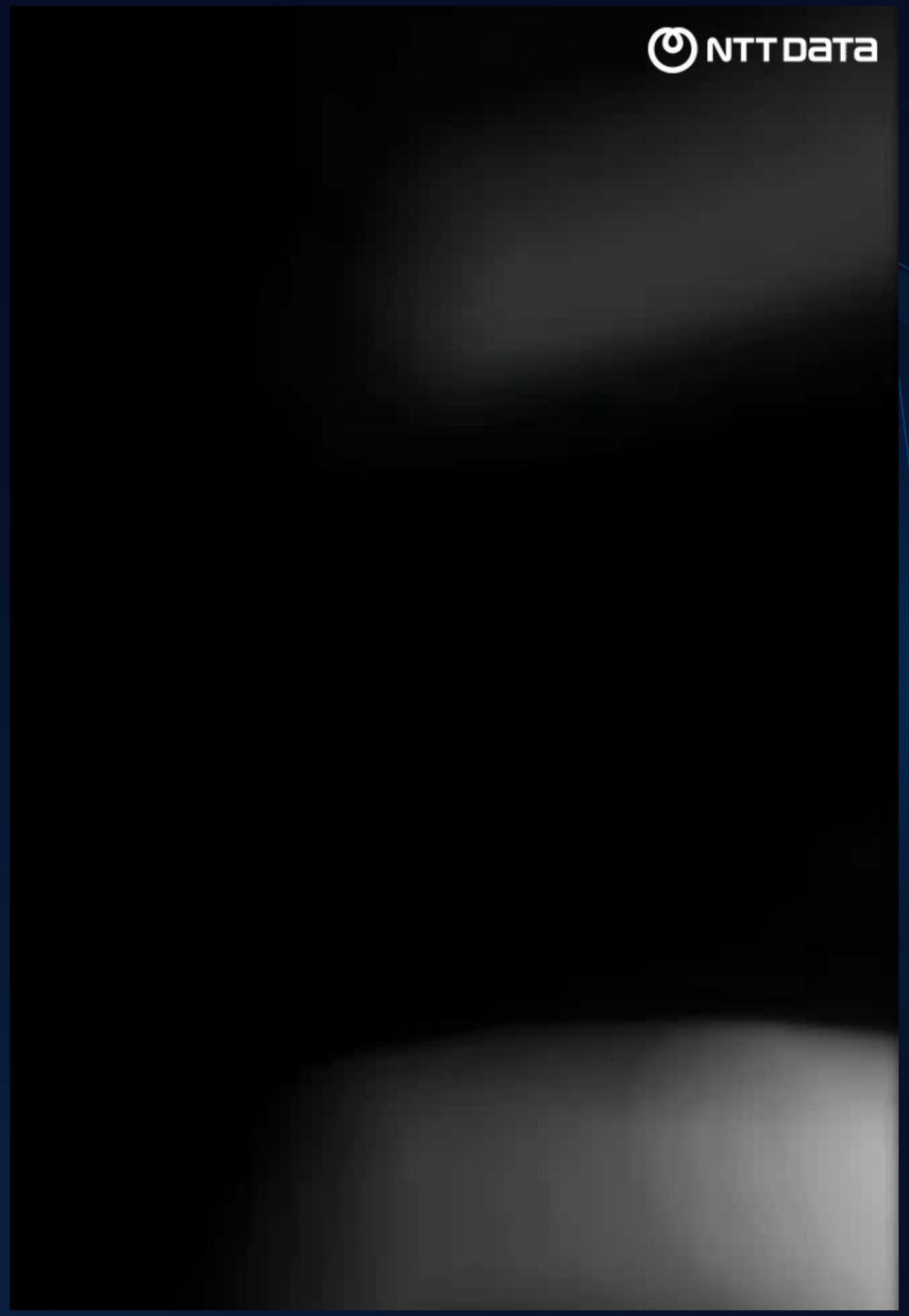
4 sesiones - Teoría + comparativa + demo o práctica guiada en cada bloque.

PROPÓSITO

Salir con un flujo trazable Issue → proposal → specs → tasks → validate

© 2026 NTT DATA, Inc.

BCP



NTT DATA

© 2026 NTT DATA, Inc.

Objetivos y *recorrido.*

Del qué al cómo, del cómo al agente, y del agente al control. Cuatro sesiones, cada una con su laboratorio.

AL TERMINAR EL DÍA PODRÁS

1. **Distinguir SDD de sus implementaciones** y explicar el rol de OpenSpec, Spec Kit y Constitution.
2. **Transformar un cambio aprobado en artefactos trazables:** proposal, specs, design, tasks y validación.
3. **Aplicar los fundamentos de agentes:** model, context, tools, loop y roles especializados.
4. **Demostrar un flujo agentic con control:** routing, evaluator, HITL y evidencia integrada en GitHub.

S1 — DEFINIR
OpenSpec, Spec Kit y Constitution
Estructurar intención, reglas y evidencia antes de implementar
45 min teoría + demo ReqFlow · **Lab 1 · 50 min · Definición técnica + contrato**

S2 — TRAZAR
Specify → Plan → Tasks → Validate
Issue → proposal/specs → design/tasks → validate
40 min teoría · **Lab 2 · 60 min · Spec-to-tasks**

S3 — DELEGAR
Fundamentos de agentes
Single-agent: model + instructions + context + tools + loop
45 min teoría · **Lab 3 · 70 min · Single-agent**

S4 — CONTROLAR
Routing, Evaluator y HITL
Agents + MCP + OpenSpec + review gates
35 min teoría · **Lab 4 · 90 min · Agente + review gate**

META DEL DÍA **Orquestar un cambio asistido por agentes, con controles humanos, validación automática y trazabilidad en GitHub.**

© 2026 NTT DATA, Inc.

SECCIÓN 01

OpenSpec, Spec Kit y Constitution.

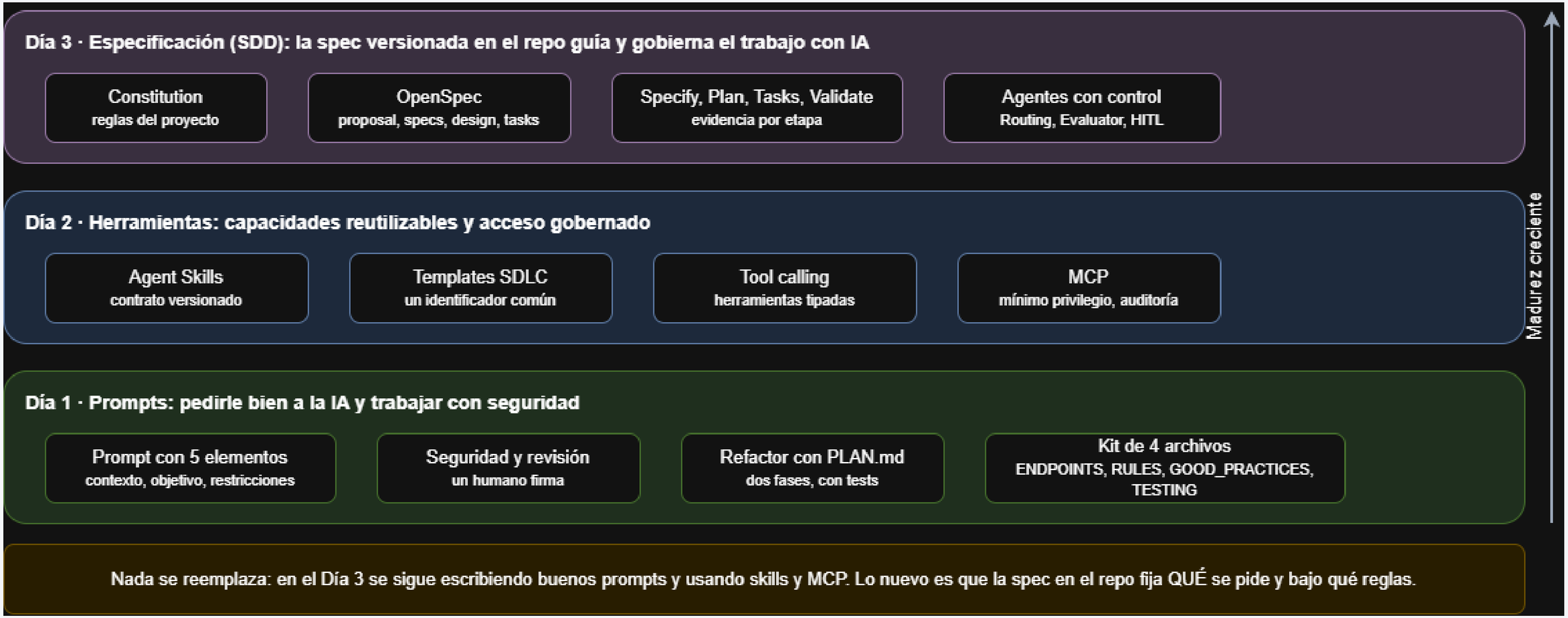
Estructurar intención, reglas y evidencia antes de implementar

Definir: si el agente no sabe qué está permitido, improvisa.

01

© 2026 NTT DATA, Inc.

Cómo evoluciona el uso de la IA: cada capa se suma, no reemplaza



© 2026 NTT DATA, Inc.

¿Qué es *Spec-Driven Development?*

Disciplina para definir cambios de forma entendible por humanos y verificable durante su ejecución.

PREGUNTA GUÍA *¿De dónde sale la verdad de un cambio: del prompt o de la spec?*

EL PROBLEMA

Vibe coding

- Requisitos en Word que se desactualizan.
- Ambigüedad resuelta inventando soluciones.
- Intención re-explicada en cada prompt.

LA SOLUCIÓN

Spec-Driven Development

- Specs versionadas junto al código.
- Criterios verificables y evidencia de cumplimiento.
- Fuente de verdad compartida humano/agente.

Intención → **Spec versionada** → **Agente / equipo** → **Evidencia**

LO CLAVE **La spec es el contrato: el prompt cambia en cada sesión, la spec queda en el repositorio.**

© 2026 NTT DATA, Inc.

Por qué SDD *importa.*

Disciplina para definir cambios de forma legible para humanos y verificable para el agente. La diferencia entre re-explicar la intención en cada prompt y dejar que la spec la sostenga.

EL PROBLEMA · "VIBE CODING"

Re-explicar intención en cada prompt.

Requisitos en Word que se desactualizan al primer merge.

Ambigüedad que el modelo "resuelve" inventando un supuesto.

Tickets donde la intención se re-formatea cada vez y el contrato cambia.

Sin fuente de verdad compartida entre humano y agente.

Decisiones técnicas y de negocio mezcladas en el mismo párrafo.

Síntoma visible: cada PR es una negociación, no una confirmación.

LA SOLUCIÓN · SDD

El cambio se describe antes de codificarse.

Specs versionadas junto al código, en el mismo pull request.

Criterios verificables y evidencia de cumplimiento por etapa.

Fuente de verdad compartida entre humanos, agentes y auditoría.

Cambio dividido en artefactos pequeños, firmables y revisables.

Reglas explícitas que el agente no tiene que asumir.

Síntoma visible: cada PR cierra con evidencia escrita, no con capturas.

REGLA **Una spec vive hasta que la evidencia la desmiente o hasta que un humano la depreca.**

© 2026 NTT DATA, Inc.

Del *vibe* al *contrato.*

Cuatro formas de producir software que aparecen en cualquier equipo de BCP. La prácticas maduras se mueven hacia la derecha conforme el cambio crece o toca superficies sensibles.

01 VIBE

Prompt creativo.

Útil para explorar ideas y nombres.
Riesgoso en código regulado o sensible.
No deja huella verificable.

USO · EXPLORACIÓN

02 HISTORIA

Ticket con criterios.

Historia de usuario con criterios.
Sirve para sprints, no para cambios de plataforma.
Pierde vigencia si nadie lo revisa.

USO · SPRINTS

03 SPEC

Cambio con artefactos.

Proposal, specs, design y tasks versionados.
Fuente única, criterio verificable, evidencia al PR.
Ya del lado correcto para BCP.

USO · DEFAULT

04 CONTRACT

Spec + agente gobernado.

Specs + gates + agente con roles.
Aplica para cambios con auditoría, contratos o Plataforma.
Evaluable continuamente, no solo al final.

USO · PLATAFORMA

MODOS 1 Y 2

Válidos para explorar y avanzar sprints.

Sirven cuando el cambio es chico y el equipo es chico. No escalan cuando el output cruza contratos, auditores o automatización.

MODOS 3 Y 4

El lugar al que un banco debe llegar por defecto.

Spec + contract no es burocracia: es lo que le da al equipo la opción de meter agentes sin perder trazabilidad.

© 2026 NTT DATA, Inc.

Tres enfoques, *tres roles.*

Hacer explícitas las reglas y restricciones que el agente debe respetar. No son competidores: cada uno cubre una capa distinta del mismo problema.

MARCO

OpenSpec

Framework ligero para estructurar cambios a través de **proposal, specs, design, tasks y lifecycle**. Lo usamos como mecanismo principal del laboratorio.

Aporta: artefactos versionados, transiciones explícitas, ASCII legible para quien revisa el PR.

CAPA · ARTEFACTOS DEL CAMBIO

TOOLKIT

Spec Kit

Toolkit para **implementar SDD con artefactos y comandos guiados** como *specify* y *plan*. Genera esqueletos de specs y planes para acelerar al equipo.

Aporta: comandos para iniciar el cambio, convenciones de carpeta, plantillas reutilizables.

CAPA · FLUJO GUIADO

REGLAS

Constitution

Conjunto de **principios y reglas del proyecto** que orientan specs, planes y decisiones. El documento más estable del repo.

Aporta: lista cerrada de restricciones no negociables, evidencia explícita para cada excepción, versionada junto al código.

CAPA · PRINCIPIOS VERSIONADOS

CÓMO ENCAJAN

Constitution (reglas) → **OpenSpec** (cambio estructurado) → **Spec Kit** (flujo guiado) → **Agente respeta** y entrega evidencia

LO CLAVE

Constitution define qué está permitido; OpenSpec y Spec Kit organizan cómo se pide y se ejecuta el cambio.

© 2026 NTT DATA, Inc.

Estructura típica *de Constitution.*

Cinco bloques para convertir políticas en reglas verificables.

PREGUNTA GUÍA *¿Qué regla de tu equipo cabe en cada bloque?*

1 Arquitectura	2 Seguridad	3 Calidad	4 Gobierno	5 Excepciones
Capas, contratos, dependencias permitidas.	Manejo de secretos, autenticación, autorización y controles requeridos.	Cobertura mínima, CI obligatorio, quality gates.	Reglas de aprobación, hitos de revisión humana.	Proceso y evidencia para saltar una regla.

QUÉ CUBRE

De lo estructural (arquitectura, seguridad, calidad) a lo organizacional (gobierno y excepciones): qué se permite, quién aprueba y cómo se documenta una excepción.

EN ESTE LABORATORIO

No implementamos Spec Kit; usamos esta estructura como referencia conceptual y trabajamos OpenSpec como mecanismo principal.

LO CLAVE **Una Constitution mínima y con reglas verificables vale más que un manual extenso que nadie consulta.**

© 2026 NTT DATA, Inc.

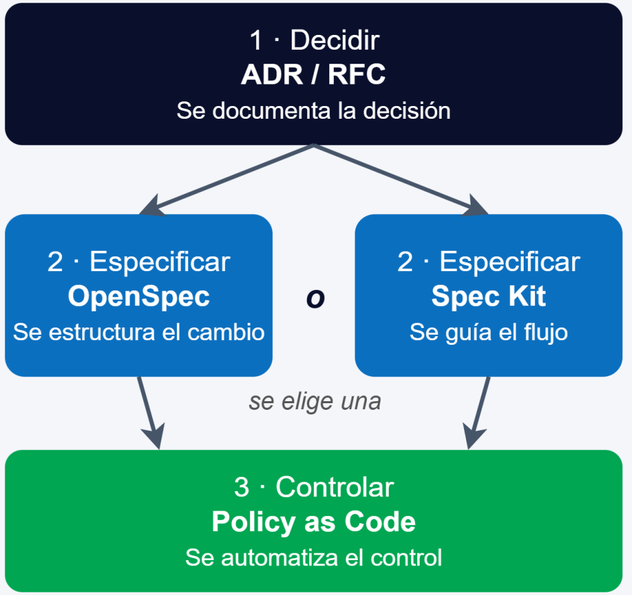
Herramientas y enfoques *complementarios.*

No compiten: cada uno resuelve una parte del problema.

PREGUNTA GUÍA *¿Cuál uso para qué momento del cambio?*

ENFOQUE	PARA QUÉ SIRVE	EN EL CICLO
OpenSpec	Estructura cada cambio como propuesta incremental sobre una spec existente	Especificar
Spec Kit	Guía el flujo completo desde cero: constitución, spec, plan, tareas e implementación	Especificar
ADR / RFC	Registra el porqué de las decisiones arquitectónicas y sus alternativas	Decidir
Policy as Code	Verifica de forma automática las reglas de cumplimiento	Controlar

DÓNDE ENTRA CADA UNA



LO CLAVE Cada herramienta ocupa un momento distinto del ciclo. OpenSpec y Spec Kit resuelven el mismo momento (especificar): se elige una como fuente de verdad.

© 2026 NTT DATA, Inc.

SECCIÓN 02

Specify, Plan, Tasks y Validate.

El flujo SDD en la práctica

Trazar: cada etapa deja evidencia, y cada evidencia habilita la siguiente.

02

© 2026 NTT DATA, Inc.

El pipeline SDD, *de un vistazo.*

Cada etapa responde una pregunta distinta y produce evidencia específica. Separarlas te permite auditar, repetir y paralelizar el cambio con criterio.

ETAPA 01

Specify

¿QUÉ?

Spec funcional natural, escrita como si la leyera un humano. Criterios de aceptación, sin tecnología, sin atajos.

ETAPA 02

Plan

¿CÓMO?

Plan técnico: arquitectura, riesgos, alternativas descartadas. Aquí ya se discute qué contratos se tocan.

ETAPA 03

Tasks

¿PASOS?

Descomposición atómica del cambio en tareas pequeñas, verificables y ejecutables incrementálmente.

ETAPA 04

Validate

¿EVIDENCIA?

Evidencia ejecutable: validación de specs, tests, contratos y quality gates. No es un checklist al final.

VALOR · POR QUÉ SEPARAR

Separar qué, cómo, pasos y evidencia evita saltar a implementar y permite auditar cada decisión.

01 · Evidencia por etapa — sabes qué falló antes.

02 · Rollback parcial por artefacto, no por PR.

03 · Auditoría legible sin reunión adicional.

04 · Agentes con criterios de salida claros.

© 2026 NTT DATA, Inc.

Tips de implementación *y estrategias.*

Una regla breve por etapa y tres estrategias para escalar sin perder control.

TIPS POR ETAPA

SPECIFY	Define qué debe cumplirse y cómo comprobarlo, sin decir aún cómo se programa.
PLAN	Define cómo se va a resolver, con sus riesgos y los contratos que se tocan, antes de escribir código.
TASKS	Divide el plan en pasos pequeños, cada uno con un resultado que se pueda comprobar.
VALIDATE	Comprueba con pruebas y evidencia que lo construido cumple lo que dice la spec.

ESTRATEGIAS Y REFERENCIAS

1. **Pilotear 1 HU end-to-end antes de escalar.**
2. **Versionar los artefactos del cambio junto al código.**
3. **Gate humano si el Plan toca contratos.**

REF Dan North (BDD), Jeff Patton (Story Mapping).

LO CLAVE **Un piloto end-to-end con una sola HU enseña más que diez specs sin ejecutar.**

© 2026 NTT DATA, Inc.

Entorno *Brownfield.*

Primero entender y proteger lo que existe; después cambiar.

OBSERVAR	DESCUBRIR	PRIORIZAR	FLUJO SDD
Ver el sistema y sus señales.	Producir hipótesis y evidencia.	Elegir el cambio mínimo aprobado.	Specify → Plan → Tasks → Validate.

Buenas prácticas

- Caracterización primero: proteger comportamiento actual.
- Cambios ultra-pequeños.

Una sola regla

- si una tarea no sale de la spec, no entra. Se detiene y se revisa.

LO CLAVE **En brownfield, el primer entregable es un test de caracterización, no código nuevo.**

© 2026 NTT DATA, Inc.

SECCIÓN 03

Fundamentos de Agentes.

Model, Tools, Context y Loop

Delegar: un agente es un modelo con herramientas, contexto y un ciclo de trabajo.



03

© 2026 NTT DATA, Inc.

Cuatro componentes, *cuatro modos de fallar.*

Un agente es la combinación de cuatro elementos. Si el agente falla, casi siempre hay que mirar primero cuál de estos cuatro está mal calibrado.

COMPONENTE 01

Model

El modelo que interpreta contexto y decide o genera la siguiente acción. Es el único componente del que normalmente no tienes control fino.

FALLA SI · MODELO DÉBIL O NO ACORDE A LA TAREA

COMPONENTE 02

Tools

Capacidades externas que el agente puede invocar: una API, una consulta, un servidor MCP. Una tool mala erosiona todo el flujo.

FALLA SI · RUIDOSAS O NO DETERMINÍSTICAS

COMPONENTE 03

Context

Lo que el modelo "ve" en cada paso: instrucciones, historia, documentos y salidas previas. Es finito y se llena rápido.

FALLA POR · SATURACIÓN DEL CONTEXTO

COMPONENTE 04

Loop

El ciclo razonar → actuar → observar. Si no hay criterio de éxito, condición de salida o tope, el agente itera indefinidamente.

FALLA SI · ITERA SIN CONVERGER

REGLA DIAGNÓSTICA

Cuando un agente falla, recorre estos cuatro en orden.

¿Model puede con la tarea? → ¿Tools devuelven bien? → ¿El contexto está completo y no saturado? → ¿El loop tiene criterio de salida?

© 2026 NTT DATA, Inc.

Conceptos *que orbitan* al agente.

Cuatro términos aparecen una y otra vez. Cada uno resuelve un subproblema distinto del mismo modelo; entenderlos evita confundirlos con sinónimos.

PATRÓN REACT

Alterna pensar y actuar.

El agente razona sobre qué hacer, ejecuta una tool, observa el resultado y vuelve a razonar. Es la implementación más simple del Loop. Funciona bien para tareas acotadas con observaciones legibles.

TOOLING TOOL CALLING

Cómo se invocan las tools.

El agente recibe una descripción de tools y devuelve llamadas en JSON estructurado. La implementación concreta del componente Tools. Define verbos, argumentos esperados y errores posibles.

CONTEXTO RAG

Cómo se alimenta el contexto.

Recuperación + generación: traer los documentos o tickets relevantes antes de pedirle al modelo que decida. RAG no reemplaza al buen prompting, lo extiende.

PROTOCOLO MCP

Tools interoperables.

Model Context Protocol: una forma estándar de exponer tools a un agente. Hace que conectar un agente a un servidor nuevo no requiera reescribir el cliente.

APLICACIONES

Single-agent para una tarea acotada del SDLC	Agente con tools GitHub vía MCP.
Code review	Evalúa el diff contra criterios, specs y reglas del proyecto.

NUNCA OLVIDAR

- El agente vive gracias al loop. Si el loop no converge, ninguna cantidad de contexto o de tools lo salva.
- No son tecnologías rivales: ReAct es el ciclo, MCP y Tool Calling son las manos, RAG alimenta la memoria.

© 2026 NTT DATA, Inc.

Tips de implementación *y estrategias.*

Menos es más: modelo adecuado, tools mínimas, contexto limpio.

TIPS DE IMPLEMENTACIÓN

- **Elegir modelo según complejidad.**
- **Describir tools con verbos exactos.**
- **No sobrecargar contexto en el loop.**
- **Definir éxito medible de salida.**

ESTRATEGIAS Y REFERENCIAS

1. **Empezar con el conjunto mínimo de tools necesario.**
2. **Si el loop se degrada, reducir o reorganizar el contexto.**
3. **Probar con inputs adversariales.**

REFERENCIA

Anthropic · Building Effective Agents.

LO CLAVE **Empieza simple: el agente más pequeño que resuelve la tarea es el más fácil de gobernar.**

© 2026 NTT DATA, Inc.

Roles y permisos *(taller).*

Cuatro roles operativos; ver una tool no equivale a poder usarla.

ROLES MCP OPERATIVOS

PM
PM
Prioridad, Specs

DI
Discovery
Hallazgos, Backlog

AR
Architect
Límites, Design, Tasks

DV
Dev
Ejecución acotada

FLUJO DEL CAMBIO

Cada rol toca artefactos distintos: el PM fija prioridad y specs; Discovery aporta evidencia y backlog; el Architect define límites, design y tasks; el Dev ejecuta de forma acotada.

CAPACIDAD ≠ AUTORIZACIÓN

Veo la tool ≠ **Tengo permiso**

- Ver una tool no garantiza permiso.
- Requiere credenciales y scopes.
- En ejecución desde GitHub Copilot Agents pueden aparecer ramas copilot/*; siguen sujetas a revisión humana y a los mismos criterios de evidencia.

LO CLAVE **Los permisos se otorgan por rol y por scope; cualquier rama de un agente pasa por la misma revisión humana.**

© 2026 NTT DATA, Inc.

Cuándo *NO* usar un agente.

Poner un agente donde no toca suele ser más caro que no ponerlo. Aquí van las tres preguntas que un equipo de BCP puede hacerse antes de aprobar un flujo agentic.

PREGUNTA 1

¿La salida es determinística?

Si la forma correcta es siempre la misma y se puede escribir un test, probablemente un agente sobra. Un loop que itera hasta converger termina costando más que el script que hace lo mismo en una línea.

Ejemplo — mapear un campo de un JSON conocido a otro campo de un JSON conocido. No es un trabajo de agente, es una función.

PREGUNTA 2

¿El costo del error es alto?

Si una mala decisión es irreversible o costosa (movimiento de dinero, autorización, dato personal), el agente no decide solo: enruta a un humano, sin excepciones académicas.

PREGUNTA 3

¿Hay evidencia suficiente?

Si el bucle termina sin señal clara de cierre, el agente va a iterar por inercia. Mejor esperar a tener un test de aceptación o un observable verificable.

LA REGLA DEL NUDO

Si dudas, empieza con scripts.

El agente entra cuando la complejidad del input o el costo de escribir la lógica empieza a escalar. Un script modular y testeado te da bases para que un agente luego — no al revés.

Un proyecto donde se empieza por un agente para terminar acoplando tests al agente es una trampa común. La inversa también.

También aplica al revés: cuando el input es ruidoso o hay demasiadas variables, un script se rompe y un agente que razona puede encontrar el patrón.

PREGUNTA DE CIERRE ¿Este problema existiría sin agente? Si sí, el agente amplifica lo bueno y lo malo igual.

© 2026 NTT DATA, Inc.

SECCIÓN 04

Patrones de Agentes.

Routing, Evaluator y HITL

Controlar: autonomía con evidencia, decisión humana donde importa.

04

© 2026 NTT DATA, Inc.

Patrones *bancarios core.*

Separar responsabilidades y decisiones para hacer el flujo verificable.

PREGUNTA GUÍA *¿Quién decide, quién verifica y quién aprueba?*

DECIDE EL CAMINO

Routing

- Dirige solicitudes al flujo correcto.
- Aísla tareas sensibles (auth).

¿Por dónde va?

VERIFICA LA SALIDA

Evaluator

- Evalúa una salida contra criterios explícitos antes de avanzar.
- Puede ser determinístico (tests/checks) o basado en modelo.

¿Cumple?

APRUEBA UN HUMANO

HITL

- Control humano explícito.
- Recomendado en cambios sensibles, contratos y acciones irreversibles.

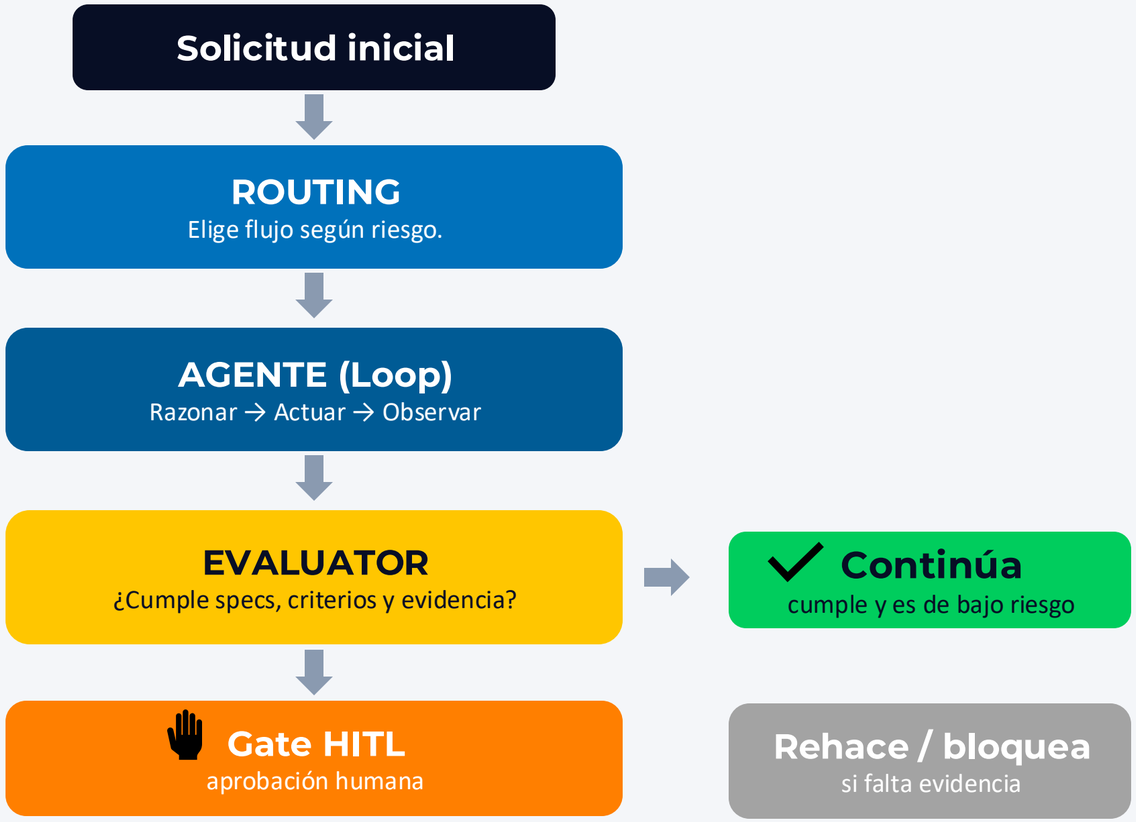
¿Lo autorizo?

LO CLAVE **Tres preguntas, tres patrones: por dónde va, si cumple y si un humano lo autoriza.**

© 2026 NTT DATA, Inc.

Combinación *de patrones.*

Así se encadenan en un solo flujo controlado.



CÓMO LEERLO

- Routing separa lo rutinario de lo sensible antes de que el agente actúe.
- El agente itera dentro de su loop con tools y contexto acotados.
- El Evaluator contrasta la salida con specs, criterios y evidencia exigida.
- Lo sensible pasa por un Gate HITL; lo demás continúa sin fricción.
- Si falta evidencia, el flujo vuelve a rehacer o se bloquea.

LO CLAVE **El orden importa: primero se enruta, luego se ejecuta, se evalúa y solo lo sensible llega a un humano.**

© 2026 NTT DATA, Inc.

Tips de implementación *y aplicaciones.*

Controles proporcionales al riesgo, sin fatiga de aprobación.

TIPS DE IMPLEMENTACIÓN

Evitar HITL absoluto (alert fatigue).

Evaluator requiere criterios/rúbrica explícitos y versionados.

Router debe ser explicable.

Medir falsos positivos del Evaluator.

APLICACIONES

Cambios asistidos por IA

tests/checks + revisión humana

Soporte

Routing separa FAQ vs Sensible

Docs

Evaluator verifica consistencia vs código

LO CLAVE **Más gates no es mejor gate: pocos, explicables y medidos.**

© 2026 NTT DATA, Inc.

Estrategias *y referencias.*

Decisiones de diseño que se pueden aplicar mañana.

ESTRATEGIAS

1. **Priorizar evaluator determinístico (tests).**
2. **Routing solo si hay flujos distintos.**
3. **HITL integrado en puntos de decisión o integración.**

REFERENCIAS

OpenAI Cookbook
Orchestrating

Zheng et al.
MT-Bench

Google Cloud
HITL patterns

LO CLAVE **Primero lo barato y determinístico; el juicio humano, donde la decisión lo amerita.**

© 2026 NTT DATA, Inc.

Gates *end-to-end.*

Seis aceptaciones humanas a lo largo del flujo; el Evaluator informa cada una.

GATES OBLIGATORIOS

1. Aceptar Specify
2. Aceptar Plan
3. Aceptar Tasks
4. Aceptar caracterización / baseline
5. Aceptar implementación y evidencia
6. Aceptar cierre del PR / DoD

ETAPA DONDE APLICA

Specify → Plan → Tasks → Baseline → Evidencia

SALIDAS DEL EVALUATOR

- CUMPLIDO
- PENDIENTE DE OTRO GATE
- NO APLICA AL ALCANCE
- FALTA EVIDENCIA
- BLOQUEADO

"Gates baratos temprano; evidencia más costosa después."

CIERRE DÍA 3 Cada gate se cierra con una salida del Evaluator: sin evidencia no hay aceptación. Puente al Día 4: gobernar equipos de agentes.

© 2026 NTT DATA, Inc.



© 2026 NTT DATA, Inc.